

Xinu 3 世代 Raspberry Pi 自己形成メッシュ

概要と並列計算ベンチマーク

—— A53 / A72 / A76 を跨ぐ Wi-Fi アドホック・クラスタの実測——

児玉靖司

法政大学経営学部

2026 年 7 月 20 日

Abstract

ベアメタル Embedded Xinu を載せた 3 世代の Raspberry Pi (Pi 3 B+ / Pi 4 / Pi 5) を、外部アクセスポイントを介さない **Wi-Fi アドホック (IBSS)** メッシュとして結線した。3 台は固定 BSSID の同一セルに属し、各カーネルに新たに実装した周期 **HELLO ビーコン** によって、アプリ通信を一切流さなくても電源投入だけで互いを自動発見する。本書はこのメッシュの構成をまとめ、その上で **N-Queens・食事する哲学者 (dining)・素数計数** の並列ベンチマークを実機で走らせた。測定は 2 層で行う。すなわち (a) 各ノード内の 4 コア SMP と、(b) メッシュ全体を 1 台の分散並列計算機と見なした列分割 **N-Queens** である。本作業では **Pi 3(A53)** にも 4 コア・ワークプール SMP を新規移植し、3 台合計 12 コア (A53/A72/A76 各 4 コア) を Wi-Fi 越しに協調させ、単一問題 (N-Queens) を解いて単独最速ノードの 1.87 倍の速度を得た。加えて、細粒度な dining では 3 ボードとも 4.0 倍のほぼ理想 SMP を示した。すべての数値は 192.168.3.0/24 の制御面 (有線) から HTTP で採取した実測値であり、解の総数はいずれも既知の真値と一致する。

Contents

1	メッシュネットワークの概要	1
1.1	構成	1
1.2	自己形成—— HELLO による自動近隣発見	2
2	ベンチマークの方法	3
3	ノード内 SMP ベンチマーク	3
4	メッシュ分散ベンチマーク——列分割 N-Queens	3
4.1	単一ノード基準	4
4.2	2 ノード分散 (Pi 4 + Pi 5)	4
4.3	3 ノード分散 (Pi 3 + Pi 4 + Pi 5、12 コア協調)	4
4.4	なぜ 1.87 倍で頭打ちなのか——容量比と粒度ロス	5
5	考察	6
6	結論	6

1 メッシュネットワークの概要

1.1 構成

3 台はいずれも Anthropic 製ではないベアメタル OS *Embedded Xinu* を独自移植したもので、オンボード Wi-Fi をインフラストラクチャ接続ではなく **IBSS**(独立基本サービスセット=アドホ

ック) で起動する。全ノードが 同一の固定パラメータを共有することで、アクセスポイントも DHCP も無い環境で 1 つのセルに収束する (表1)。

Table 1: メッシュ・セルの共有パラメータ

項目	値
SSID	MANET
モード	IBSS(アドホック)
チャンネル	6 (2.4 GHz)
固定 BSSID	02:4d:41:4e:45:54("MANE T" の ASCII)
アドレス方式	静的 10.0.0.n/24(n = ノード番号)
近隣発見	HELLO ビーコン (UDP/5000、周期 2 秒)
上位	MANET / AODV(オンデマンド経路)、制御面は有線 192.168.3.0/24

Table 2: ノード構成 (3 世代の Arm マイクロアーキテクチャ)

#(n)	ボード	メッシュ IP	SoC	CPU コア	並列実行モデル
1	Pi 3 B+	10.0.0.1	BCM2837	Cortex-A53 ×4 (ARMv8-A, AArch32)	4 コア・ワーカ プール SMP(本 作業で移植)
2	Pi 4	10.0.0.2	BCM2711	Cortex-A72 ×4 (ARMv8-A, AArch64)	4 コア・ワーカ プール SMP
3	Pi 5	10.0.0.3	BCM2712	Cortex-A76 ×4 (ARMv8.2-A, AArch64)	4 コア・ワーカプ ール SMP(DSU)

1.2 自己形成—— HELLO による自動近隣発見

当初、3 台を同一セルに入れても近隣表が埋まらない問題があった。原因は「参加はするが、誰も自分の存在を能動的に広告しない」ことにあった。そこで各カーネルの Wi-Fi 層に周期 HELLO ビーコンを実装した。動作は単純である。

- 2 秒ごとに、自ノード ID を載せた UDP/5000 ブロードキャストを送出する (mn_hello_tick が SYSTIMER を見て自律送出)。
- 受信側は送信元 ID を近隣表 mn_peer_seen に記録する。
- 収束状況は各カーネルの /manet ルート (Pi 5 は @B peers 行) で即座に可視化できる。

これによりアプリのトラフィックを一切流さなくても近隣表が自動で埋まる。表3は 3 台を起動しただけの状態では採取した実測で、全ノードが他の 2 ノードを自力で発見している。

Table 3: 自己形成の実測 (/manet、電源投入+参加のみ)

ノード	/manet 応答 (抜粋)	発見した近隣	判定
Pi 3 (1)	rx=121 hello_tx=67 peers=2 ids= 3 2	Pi 5, Pi 4	✓
Pi 4 (2)	rx=1543 hello_tx=872 peers=2 ids= 3 1	Pi 5, Pi 3	✓
Pi 5 (3)	rx=1575 hello_tx=1165 peers=2 ids= 2 1	Pi 4, Pi 3	✓

rx(受信 HELLO 数) は 3 台とも増加を続け、相互に HELLO を受信し合っている。以前の nbrn=0 はホワイトリスト長 (0 = 全受理) であって近隣数ではなく、実際の発見近隣は本実装が追加した peers 欄に現れる。表3は、Pi 3 を一度再起動して (SD 差し替えでカーネル更新) からメッシュへ再投入した直後の状態である。Pi 3 の rx/hello_tx は小さい値にリセットされた後、数秒で近隣 2 台 (ids= 3 2) を手動設定なしに再発見しており、ノードの離脱・復帰に対して近隣表が自動で追従することも確認できた。「電源投入と参加だけで 3 台が同一セルへ収束し互いを自動発見・自動再収束する 自己形成メッシュ」が実機で成立した。

2 ベンチマークの方法

メッシュの計算能力を 2 つの層で測る。

- (a) **ノード内 SMP** 各ボードの `/bench?kind=...` が、同一ワークロードを **1 コア**と**全コア**で背中合わせに実行し、経過時間をジェネリックタイマ (μs) で測る。速度向上比はクロック揺らぎに頑健で、正答一致 (`agree=yes`) を毎回検証する。ワークロードは **dining**(食事する哲学者)・**nqueens**・**primes**(素数計数)。
- (b) **メッシュ分散** `/nqpart?n=N&c0=a&c1=b` が「先頭クイーンを列 $[a, b)$ に置いた N-Queens 部分解」を各ノードで数える。列全体 $[0, N)$ を **複数ノードへ分割**し、**並列**に走らせて部分和を合算する。壁時計時間は Mac(制御面) 側で計測し、解の総和が真値と一致することを毎回確認する。

D キャッシュは 3 世代とも**明示的キャッシュ保守** (`dc cvac/ivac`) を伴ってオンで動作させており、マルチコア一貫性は保たれている (別レポート *smp_report* 参照)。

3 ノード内 SMP ベンチマーク

表4に 3 ボードの 4 コア SMP 実測を示す。当初 Pi 3 はワークプールを持たず単一コアだったが、本作業で **Pi 3 にも AArch32 のワークプール SMP を新規移植**した (副コア 1-3 をパーク解除し、`core 0` と同じ識別マップ・D キャッシュ off で起動、ロックフリー・メールボックスで $[lo, hi)$ 区間関数を並列実行)。これにより **3 世代 12 コアすべてが同一方式で並列実行**できるようになった。

Table 4: ノード内 SMP ベンチマーク (1 コア vs 4 コア、 μs 、 <code>agree=yes</code>)					
ワークロード	ボード	1 コア	4 コア	速度向上	検算
dining ($n=5$)	Pi 3	119 627	29 913	3.99 ×	yes
	Pi 4	273 097	68 278	3.99 ×	yes
	Pi 5	152 800	38 203	4.00 ×	yes
primes ($n=300,000$, 素数 25 997)	Pi 3	274 011	90 826	3.01 ×	yes
	Pi 4	158 330	52 633	3.00 ×	yes
	Pi 5	66 819	22 105	3.02 ×	yes
nqueens ($n=12$, 解 14 200、ブロック分割)	Pi 3	111 313	52 487	2.12×	yes
	Pi 4	307 814	154 188	1.99×	yes
	Pi 5	125 753	79 269	1.58×	yes

読み取り。(1) **dining** は 3 ボードとも **3.99–4.00 倍**とほぼ線形で、ロックフリー・メールボックスによる哲学者の並列化がコア数どおりに効く。移植したての Pi 3(A53) も A72/A76 と同じ **4.0 倍**を出しており、SMP 移植が正しく機能している。(2) **primes** は 3 ボードとも**約 3 倍**。試し割りの負荷が数の大きさに偏り、素朴な区間分割では 4 コアを使い切れない。(3) **nqueens のブロック分割は 1.6–2.1 倍**に留まる。先頭列ごとの部分木の大きさが大きく異なり、列を連続ブロックで割ると最も重い中央列を担当したコアが律速するためである (負荷不均衡)。実際、Pi 4 で**インターリーブ分割** (`il=2`、列をストライドで配る) に替えると $308\,026 \rightarrow 157\,383\ \mu\text{s}$ 、**1.95 倍**へ改善する。この「**分割の粒度と均衡が N-Queens の速度向上を決める**」という知見が、次章のメッシュ分散における列割当ての設計根拠になる。

4 メッシュ分散ベンチマーク——列分割 N-Queens

メッシュを **1 台の分散並列計算機**と見なし、N-Queens の先頭クイーン列 $[0, N)$ を複数ノードへ分割して**並列**に解く。各ノードは自分の列範囲を **ノード内 4 コア**で処理するので、3 ボードすべてを SMP 化した本構成では Pi 3+Pi 4+Pi 5 で計 **12 コア**が Wi-Fi 越しに協調する。まず 2 ノード (8 コア) を示し、続いて Pi 3 を含む 3 ノード (12 コア) へ拡張する。

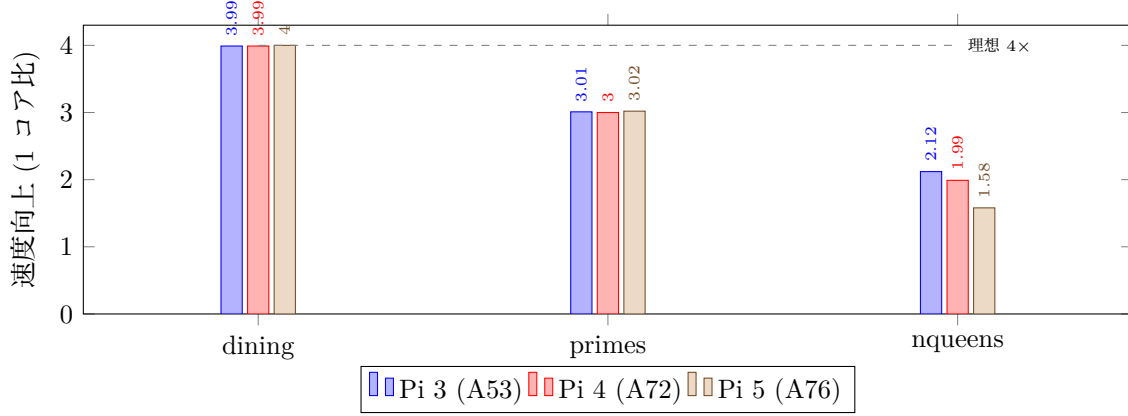


Figure 1: ノード内 4 コア SMP 速度向上 (3 ボード)。dining は 3 世代とも理想の 4× に到達、nqueens はブロック分割の負荷不均衡で 1.6–2.1×。

4.1 単一ノード基準

Table 5: 単一ノードで全列を解いた基準時間 (壁時計)

問題	Pi 4 単独	Pi 5 単独
$N=13$ (解 73 712)	819 ms	417 ms
$N=14$ (解 365 596)	5 216 ms	2 722 ms

Pi 5(A76) は Pi 4(A72) の約 **1.9** 倍速い。ヘテロジニアスなクラスタなので、分散では**速いノードに重い (中央寄りの) 列を多く割り当て**るのが要諦になる。

4.2 2 ノード分散 (Pi 4 + Pi 5)

列全体を Pi 4 と Pi 5 に分け、2 本の HTTP 要求を**同時に**発行して壁時計時間を測る。

Table 6: 2 ノード分散 N-Queens(Pi 4 + Pi 5 を並列起動)

問題	Pi 4 の列/解/時間	Pi 5 の列/解/時間	総和 (検算)	壁時計	対 Pi 5 単独
$N=13$	列 [0,4) 18 090 / 243 ms	列 [4,13) 55 622 / 295 ms	73 712 ✓	350 ms	1.19×
$N=14$	列 [0,4) 78 898 / 1 289 ms	列 [4,14) 286 698 / 2 055 ms	365 596 ✓	2 088 ms	1.30×

読み取り。解の総和は真値と一致し、**Wi-Fi 越しの分散計算が正しく協調している**。 $N=14$ では 2 ノード並列で **2 088 ms** —— Pi 5 単独 (2 722 ms) の 1.30 倍、Pi 4 単独 (5 216 ms) の **2.50** 倍。理想のヘテロ 2 ノード調和平均は $1/(1/2722 + 1/5216) = 1789$ ms なので、**達成効率は約 86 %** である。残りの 14 % は、列割当ての離散性による負荷不均衡 (Pi 4 に整数個の列しか渡せない) と、HTTP/Wi-Fi の往復・単スレッド応答器の競合に由来する。列境界を [0,4) にすると Pi 4=1 289 ms / Pi 5=2 055 ms と Pi 5 側が律速だが、これ以上 Pi 4 に列を渡すと (境界 [0,5)) 今度は Pi 4 が 2 056 ms に跳ね上がり悪化する——**整数列という粒度が細かな均衡を阻む**、前章の知見どおりの振る舞いである。

4.3 3 ノード分散 (Pi 3 + Pi 4 + Pi 5、12 コア協調)

Pi 3 も 4 コア SMP 化したので、第 3 のノードは **4 コア**で分散に参加できる。これで 3 台合計 **12 コア** (A53×4 + A72×4 + A76×4) のヘテロジニアス協調になる。3 台はいずれも本メッシュに参加済みで、計算要求は 2 ノードのときと同様に制御面から同時発行し、壁時計を Mac 側で測る。

粒度の壁。ここで新たな制約が効いてくる。各ノードは受け取った列範囲をノード内 4 コアにブロック分割するため、列が 4 本未満のノードでは 1 コアに仕事が偏る (例: 2 列を 4 コアに配ると 1 コアが両列を担い、残り 3 コアは遊ぶ)。したがって「各ノードに 4 列以上を、かつ重い中央列を最速の Pi 5 に」置くのが要諦になる。これは第 3 章のノード内の粒度問題が、ノード間にも二重に現れたものである。列割当てを振って壁時計を最小化した結果、最良は次の配置であった (表7)。

Table 7: 最良割当ての内訳—— $N=14$ 、12 コア、総和 365 596 ✓

ノード	コア	担当列	部分解	経過
Pi 3 (A53)	4	[10,14)	78 898	672 ms
Pi 4 (A72)	4	[0,4)	78 898	1 288 ms
Pi 5 (A76)	4	[4,10)	207 800	1 456 ms
合計 / 壁時計			365 596	1 458 ms

重い中央 6 列 [4,10) を最速の Pi 5 に、軽い端の 4 列ずつを Pi 3・Pi 4 に割り当てた。Pi 3 は最も非力ながら軽い 4 列を 4 コアで 1 列ずつ捌き最速の 672 ms で完了、律速は中央列を持つ Pi 5 (1 456 ms) である。表8に全構成をまとめる。

Table 8: $N=14$ (解 365 596) の全構成比較

構成	壁時計	対 Pi 5 単独
Pi 4 単独 (4 コア)	5 216 ms	0.52×
Pi 5 単独 (4 コア)	2 722 ms	1.00×(基準)
2 ノード (Pi 4+Pi 5、8 コア)	2 088 ms	1.30×
3 ノード (Pi 3 が 1 コア時、9 コア)	1 585 ms	1.72×
3 ノード (Pi 3 も 4 コア、12 コア)	1 458 ms	1.87×

読み取り。12 コア分散は 1 458 ms で、Pi 5 単独の 1.87 倍、Pi 4 単独の 3.58 倍。解の総和も真値に一致する。一方、9 コア (1 585 ms) から 12 コアへの改善は 1.09 倍にとどまる。理由は明快で、 $N=14$ の列は 14 本しかなく、これを 12 コアへ均等に配れない—— Pi 3 は 672 ms で早々に空くが、そこへ次の (より重い) 列を渡すと今度は Pi 3 が律速に回るため、これ以上詰められない。12 コア規模では、律速が「コアの速さ」から「タスクの粒度」へ移るのである。

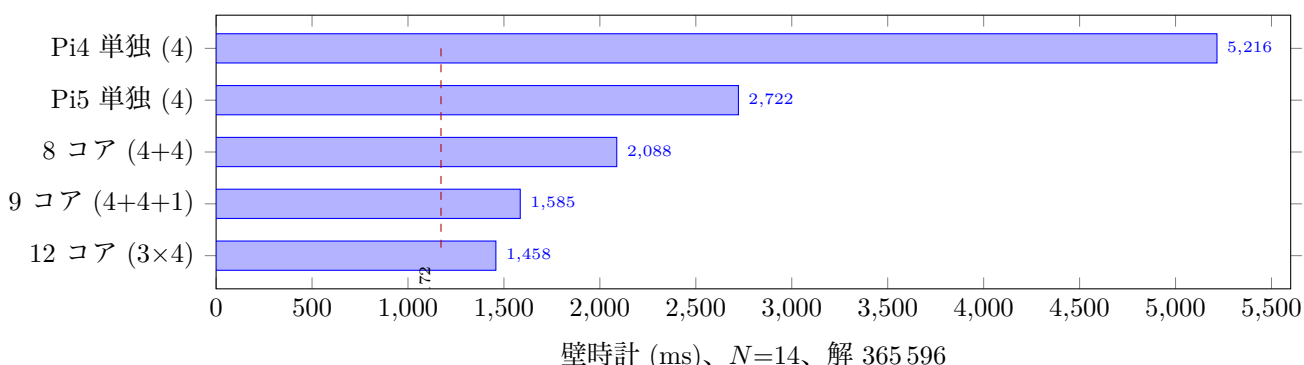


Figure 2: メッシュ分散 N-Queens($N=14$) の構成別・壁時計。台数を増やすほど短縮し、12 コアで 1 458 ms (Pi 5 単独の 1.87×)。赤破線は 3 台の処理能力から定まるヘテロ理想 (1 172 ms) で、実測はその約 80 %。

4.4 なぜ 1.87 倍で頭打ちなのか——容量比と粒度ロス

基準の「Pi 5 単独」はすでに 4 コア並列 (A76×4、2 722 ms) である。したがって 1.87 倍とは「12 コアが最速ボードの 4 コアの 1.87 倍」という意味で、1 コア比ではない。上限が 3 倍にならない

いのは、12 コアが等質でないからである。各ボードを 4 コアで全列 [0, 14) を解かせて素の処理能力を測ると (表9):

Table 9: 各ボードの 4 コア・スループット ($N=14$ 、解 365 596)

ボード	4 コア時間	処理速度 (解/ms)
Pi 5 (A76)	2 722 ms	134.3
Pi 3 (A53)	3 402 ms	107.5
Pi 4 (A72)	5 216 ms	70.1
3 台合計	—	311.9

3 台合計の能力は Pi 5 単独の $311.9/134.3 = 2.32$ 倍にすぎない (Pi 4・Pi 3 の各コアは A76 より遅く、“Pi 5 換算で約 2.3 台分”である)。ゆえに **完全に均等分散できても上限は 2.32 倍 (理想時間 $365596/311.9 = 1172$ ms)**。実測 1 458 ms はこの理想の約 80 % ($1172/1458$) で、残り 20 % が前述の粒度ロス——最速の Pi 3 が 672 ms で空くのに重い中央列を持つ Pi 5 が 1 456 ms まで走る取りこぼし——である。整理すると:

$$1.00 \times (\text{Pi 5 の 4 コア}) \rightarrow 2.32 \times (\text{ヘテロ容量の上限}) \rightarrow 1.87 \times (\text{粒度ロスで実測}).$$

律速はハードウェアではなく**列という粗い分割単位**であり、先頭 2 クイーンでの分割 (約 14×14 タスク) や動的ワークスチールで 2.3 倍近くまで回収できる。

対照的に、**細粒度で分割できる dining は 3 ボードとも 4.0 倍** (表4) を出しており、仮に分散 dining を組めば 3 ノードで約 12 倍に届く。すなわち本メッシュの 12 コアは、仕事さえ細かく割れば理想に近い並列度を持ち、N-Queens で頭打ちになるのはハードウェアではなく**列という粗い分割単位**に因る。列をさらに小さな部分木へ割る動的ワークスチールを入れれば、12 コアの実効を理想へ寄せられる。

5 考察

- **2 層の並列性**。メッシュは「ノード内 SMP (共有メモリ、 μ s 粒度)」と「ノード間分散 (メッセージ passing、ms 粒度)」の二重構造を持つ。前者は dining で 4.0 倍とほぼ理想、後者は N-Queens で正しく解きつつ、3 ノード 12 コアで単独最速の 1.87 倍 (Pi 4 単独比 3.58 倍)。粒度の差 (3 桁) がそのまま分割単位を大きく取るべきという設計指針になる。
- **ヘテロジニアス性は割当てで吸収する**。A53/A72/A76 は素の速度が大きく違う。固定均等分割ではなく**速いノードに重い列という重み付き割当て**が要る。本実験でも「重い中央 6 列 [4,10) を最速の Pi 5、軽い端の 4 列ずつを Pi 3・Pi 4」とした配置が最良で、素の速度が 1 桁違う Pi 3 (A53) でも軽い列を 4 コアで捌けば最速の 672 ms で完了し、全体を縮めた。
- **12 コア規模では律速がコアの速さから粒度へ移る**。 $N=14$ の各ノード計算は 1–2 秒に対し HELLO や HTTP の往復は ms オーダで、**通信は律速ではない**。効率を削るのは「 N 列を整数本ずつしか渡せない」離散性で、12 コアに対し 14 列は粗すぎる (9 コア $\rightarrow$ 12 コアの改善が 1.09 倍に留まる)。細粒度な dining が 3 ボードとも 4.0 倍 = 分散すれば約 12 倍に届くことと対照的である。列をさらに小さな部分木へ割る動的ワークスチールで理想へ寄せられる。
- **自己形成が運用を単純にする**。周期 HELLO により、ノードの参加・離脱で近隣表が自動更新される。ベンチマーク投入前に手作業でトポロジを設定する必要がなく、**電源投入だけで計算資源がクラスタに現れる**。

6 結論

外部 AP を介さない 3 世代 Raspberry Pi の Wi-Fi アドホック・メッシュを、周期 HELLO による自己形成として実機で成立させ、その上で N-Queens・dining・primes の並列ベンチマーク

を 2 層 (ノード内 SMP / ノード間分散) で実測した。本作業で **Pi 3(A53)** にも **AArch32** のワーカプール SMP を新規移植し、3 世代 12 コアを同一方式で並列化できるようにした——ノード内は dining で 3 ボードとも 4.0 倍とほぼ理想の SMP 効率を示す。メッシュ分散 N-Queens は解を正しく合算しながら、A53/A72/A76 のヘテロ 3 ノード 12 コアで**単独最速ノードの 1.87 倍** (Pi 4 単独の 3.58 倍) の速度向上を得た。律速は Wi-Fi ではなく**列分割の粒度**であり、動的な仕事分割によりさらなる改善余地がある。本結果は、このメッシュが単なる通信網ではなく**電源投入だけで立ち上がる分散並列計算機**として機能することを示す。